Advanced Data Structures & Algorithm Analysis (2501CS10)

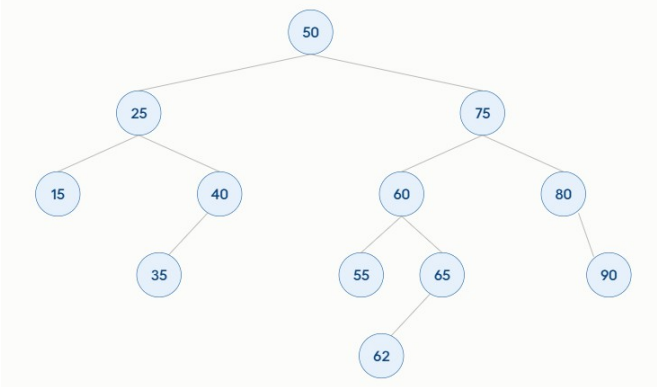
Unit-wise Important Questions

UNIT-I:

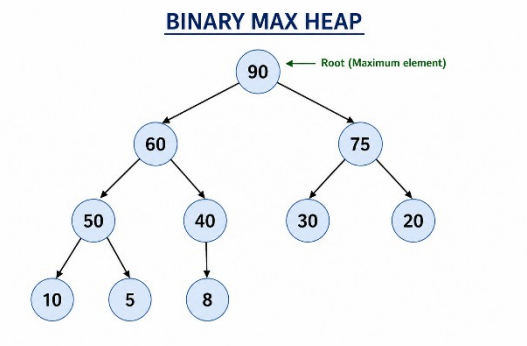
1. Explain the importance of algorithm analysis. Differentiate between time complexity and space complexity with suitable examples.
2. Describe the asymptotic notations **O, Ω, and Θ**. Illustrate each notation with suitable examples.
3. Analyze the time complexity for matrix multiplication algorithm (tabular method) using asymptotic notation and justify your answer.
4. Explain the concept of hashing. Discuss different hash functions with suitable examples.
5. Explain the collision resolution techniques Linear Probing, Quadratic Probing, Double Hashing, and Separate Chaining.
6. Differentiate between Static Hashing and Dynamic Hashing. Explain the working of Dynamic Hashing (Extendable hashing) with an example.

UNIT-II:

1. Construct an AVL Tree by inserting the following keys: 50, 25, 12, 80, 90, 37, 45, 18, 19, 6. Show the required rotations.
2. Explain the four types of AVL Tree rotations with suitable examples. Discuss the applications of AVL Trees.
3. Explain the deletion procedure in AVL tree. Show the balancing operations after deletion of 15 from the below AVL tree.



4. Construct a B-Tree of order 4 by inserting the following keys: 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8.
5. Explain the deletion operation in a B-Tree with a suitable example, and applications of B-Trees.
6. Construct a Min/Max Heap by inserting the following elements: 40, 15, 30, 10, 12, 6, 25, 18. Show all intermediate steps.
7. Perform one deletion operation on given Max Heap and show the step-by-step process for final Max Heap.



8. Explain the basic operations of a Binary Heap with their time complexities and also discuss the applications of Priority Queues.

Answers

1.Explain the importance of algorithm analysis. Differentiate between time complexity and space complexity with suitable examples.

Algorithm : An algorithm is a step-by-step set of instructions to solve a problem or perform a specific task. It must be clear, finite, and effective.

Complexity/ Algorithm Analysis: Used to evaluate the efficiency of algorithms. Measured in terms of input size n.

1.Time Complexity: How fast an algorithm runs. It measures how the runtime grows with input size n.

Time Complexity: 1 Statement/Operation = 1 Unit Time

Example 1: Simple Loop

```
for (int i = 0; i < n; i++) {
   sum += i;
}
```

Initialization: int i = 0 ⟶1 unit

Condition check: i < n ⟶ 1 unit per iteration ⟶ total=n+1

Increment: i++ ⟶ 1 unit per iteration ⟶ total=n

Body: sum += i ⟶ 1 unit per iteration ⟶total=n

Total Time: 1 + n(condition) + n(increment) + n(body) = 2 + 3n ⟶ O(n)

2.Space Complexity: How much memory it uses. The total memory required by an algorithm, including input data, temporary variables, and recursion stack with input size n.

Space Complexity = Fixed part (constants, program code, variables)+Variable part (depends on input size)

Space Complexity: 1 Assignment = 1 Unit Space

Example 1: Variable Declaration

int a = 5, b = 10;
2 assignments → 2 units of space →O(1)

Example 2: Static Array Allocation

int arr[100];
100 assignments →100 units of space →O(1)

Example 3: Variable Array Allocation

int arr[n];

n assignments → n units of space → O(n)

Importance of Algorithm Analysis

Performance prediction: Determines how an algorithm scales with input size without running it on every possible dataset.
Resource optimization: Helps balance CPU time and memory usage.
Comparative evaluation: Allows us to choose the best algorithm among multiple correct solutions.
Scalability assurance: Ensures solutions remain efficient for large inputs (e.g., millions of records).
Cost-effectiveness: Reduces hardware requirements and execution costs.

Time Complexity vs Space Complexity

Aspect	Time Complexity	Space Complexity
Definition	Measures number of computational steps relative to input size.	Measures memory required during execution.
Unit of Measure	CPU operations (growth rate of steps).	RAM consumption (variables, stack, auxiliary structures).
Notation	$O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$, $O(2^n)$…	$O(1)$, $O(n)$, $O(n^2)$…
Example 1	**Linear Search**: Checking each element → $O(n)$ time, $O(1)$ space.	Same example uses constant memory.
Example 2	**Binary Search**: Cuts problem in half → $O(\log n)$ time, $O(1)$ space.	Minimal memory usage.
Example 3	**Recursive Factorial**: $O(n)$ time due to n calls.	$O(n)$ space (stack frames).
Example 4	**Matrix Creation**: Time $O(n^2)$ to fill values.	Space $O(n^2)$ to store matrix.

2. Describe the asymptotic notations **O, Ω, and Θ**. Illustrate each notation with suitable examples.

Asymptotic Notations:

Asymptotic Notations are mathematical tools used to analyze the performance of algorithms by understanding how their efficiency changes as the input size grows.

There are mainly three asymptotic notations:

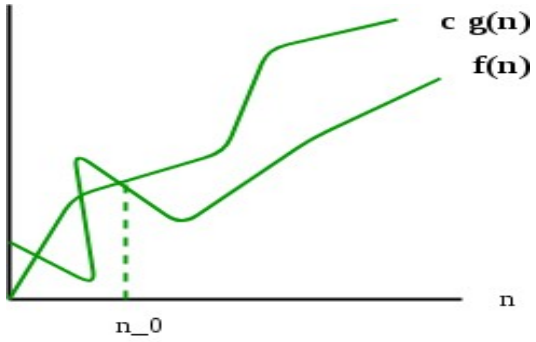
1. Big-O Notation (O-notation)
2. Omega Notation (Ω-notation)
3. Theta Notation (Θ-notation)

Big-O Notation (O-notation):

Big O gives an upper bound on the growth rate of an algorithm. It tells us the maximum time or space the algorithm could take.

If f(n) describes the running time of an algorithm, f(n) is O(g(n)) if there exist a positive constant C and n0 such that, $0 \leq f(n) \leq cg(n)$ for all $n \geq n0$

It returns the highest possible output value (big-O)for a given input.



f(n) = O(g(n))

proof : f(n) = 3n + 2 is **O(n)**

f(n) = 3n + 2

g(n) = n

We need to find constants c and n0 such that $3n+2 \leq cn$

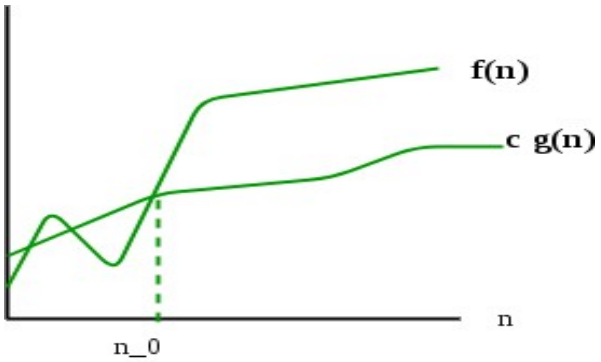
choose c=4 then no=2

Verify For every $n \geq 2$, $3n+2 \leq 4n$ Hence, $3n+2=O(n)$

Omega Notation (Ω-Notation):

Omega gives an lower bound on the growth rate of an algorithm. It tells us the minimum time or space the algorithm could take.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be **Ω**(g), if there is a constant c > 0 and a natural number n0 such that c*g(n) ≤ f(n) for all n ≥ n0



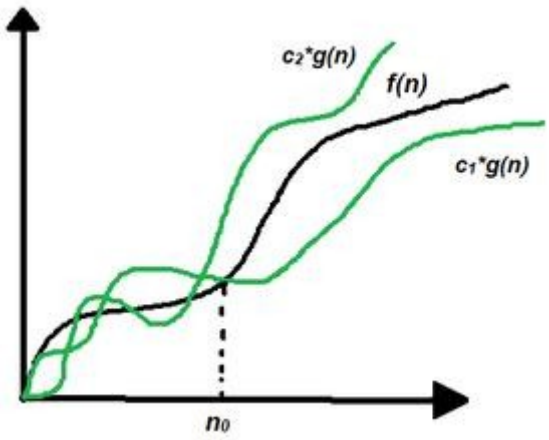
f(n) = Omega(g(n))

proof: f(n) = $5n^2 + 10$ is $\Omega(n^2)$
f(n) = $5n^2 + 10$
g(n) = n^2
choose c=5 then no=1
$5n^2 <= 5n^2 + 10$ holds true for all n >=1
so f(n)=Ω(g(n))

Theta Notation (Θ-Notation):

Theta gives an average bound on the growth rate of an algorithm. It tells us the average time or space the algorithm could take.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be Θ(g), if there are constants c1, c2 > 0 and a natural number n0 such that c1* g(n) ≤ f(n) ≤ c2 * g(n) for all n ≥ n0



proof:
f(n) = 4n - 3 is Θ(n)
c1. n <= 4n - 3<= c2. n
pick c1=1, c2=4 and n0=1 which holds true for all n>=n0

3. Analyze the time complexity for matrix multiplication algorithm (tabular method) using asymptotic notation and justify your answer.

Given two matrices **A** of dimensions m × n and **B** of dimensions n × p, the standard matrix multiplication algorithm computes the product matrix **C = A × B** of dimensions m × p.

The algorithm requires calculating the dot product of rows from and columns from .

The standard tabular method for multiplying two matrices A(of size n) and B (of size n) produces a resulting matrix C (of size n). The algorithm requires calculating the dot product of rows from and columns from .

Pseudocode

```
for i from 1 to n:
    for j from 1 to n:
        C[i][j] = 0
        for k from 1 to n:
            // Constant time mathematical operation
            C[i][j] = C[i][j] + (A[i][k] * B[k][j])
```

We analyze each statement by counting how many times it is executed.

Statement	Frequency	Cost	Total
Initialization: i = 1	1	1	1
Outer loop condition: i <= n	n + 1	1	(n + 1)
Middle loop initialization: j = 1	n	1	n
Middle loop condition: j < =n	n(n + 1)	1	n(n + 1)
Inner loop initialization: k = 1	n^2	1	n^2
Inner loop condition: k <= n	n^2 (n + 1)	1	n^2 (n + 1)
C[i][j] = C[i][j] + A[i][k] × B[k][j]	n^3	1	n^3

$T(n) = 1 + (n+1) + n + n(n+1) + n^2 + n^2(n+1) + n^3$

5. Finding the Dominant Term

Expand the expression and group terms according to their powers of n.

$$T(n) = n^3 + 3n^2 + 4n$$

Among all these terms, the highest-order term is:

$$n^3$$

During asymptotic analysis, constant coefficients and lower-order terms are ignored.

Therefore:

$T(n) = \Theta(n^3)$

4. Explain the concept of hashing. Discuss different hash functions with suitable examples.

Hashing is a technique used to store and retrieve data quickly using a special function called a **hash function** to convert a given **key** into an **index** in a **hash table**.

Instead of searching data one by one (like linear search), hashing calculates an **index/address** directly from the key.

Idea:

Suppose you want to store student records by roll number.
Instead of storing them sequentially, hashing uses:

`index=h(key)`

Where:

- `key` → the data identifier (like roll number)
- `h()` → hash function
- `index` → location where data is stored

Example:

`h(k)=k mod m`

`Let us choose m=10 then h(k)=k mod 10`

`h(25) = 25 % 10 = 5`

So the record with key `25` is stored at index `5`

Common Hash Functions:

A **hash function** converts a key into an index/address in a hash table.
General form: `h(key) = index`

A good hash function should:

- Be fast
- Distribute keys uniformly
- Reduce collisions
- Produce deterministic output

1. Division (Remainder) Method: Most commonly used method.

Formula: $h(k) = k \bmod m$

Where:

- k = key
- m = size of hash table

Example

```
m = 10
k = 37

h(37) = 37 % 10 = 7
```

2. Multiplication Method: Uses multiplication with a constant fractional value.

Formula: $h(k) = \lfloor m(kA \bmod 1) \rfloor$

Where:

- A = constant between 0 and 1
- m = table size

Example

Suppose:

```
m = 16
A = 0.618
k = 25
25 × 0.618 = 15.45
Fractional part = 0.45

16 × 0.45 = 7.2

Index = 7
```

3. Mid-Square Method: Square the key and take middle digits/bits.

Steps

1. Square the key
2. Extract middle digits
3. Use them as index

Example

```
k = 25
```

$25^2 = 625$

```
Middle digit = 2
```

Index = 2 % Table_Size

4. Folding Method: Split key into equal parts and add them.

Example

```
Key = 123456
```

```
12 + 34 + 56 = 102 % table_size
```

Types

a) Shift Folding

Simply add parts.

b) Boundary Folding

Reverse alternate parts before adding.

Example: `12 + 43 + 56=111 % table_size`

5. Digit Extraction Method: Specific digits of the key are selected to form the hash value.

Example: Key = 987654, Choose the 2nd, 4th, and 6th digits: 8,6,4

Hash Value = 864 `% table_size`

6. String Hash Functions

ASCII Sum Method

Add ASCII values.

Example:

```
"ABC"
```

```
65 + 66 + 67 = 198 % Table_Size
```

5. Explain the collision resolution techniques Linear Probing, Quadratic Probing, Double Hashing, and Separate Chaining.

Collision in Static Hashing.

A collision occurs when two different keys produce the same hash value and try to occupy the same index in the hash table.

When **two keys generate the same index**, it causes conflict.

Example:

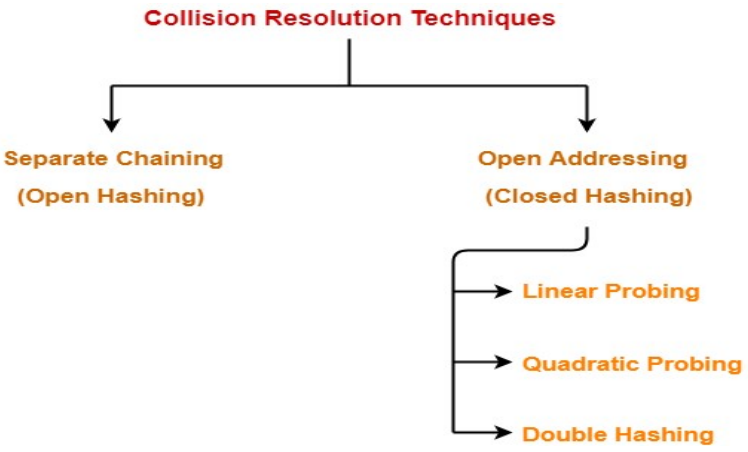
12 % 10 = 2

22 % 10 = 2 → Collision

Collision Handling Techniques:

To handle Collision, we use:

- 1. Seperate Chaining (Linked List method)
- 2. Open addressing (linear probing, quadratic probing,double hashing)



1. Separate Chaining: Each index stores a linked list of elements that hash to the same index.

Each table index stores a **linked list (chain)**.

If many keys hash to the same index, they are placed in that list.

Example:

Index 5 → 25 → 15 → 35 (linked list)

Advantages:

✔ Simple

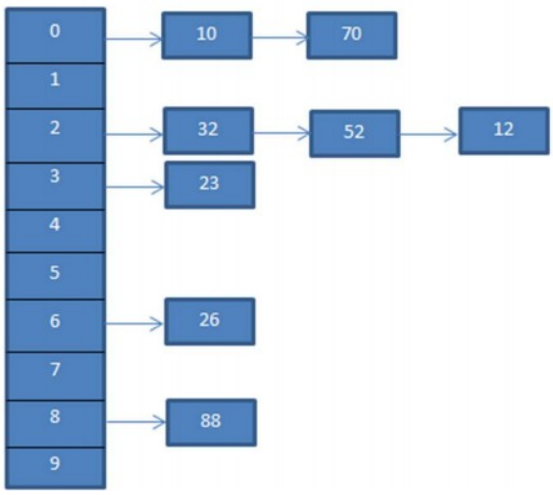
✔ No clustering

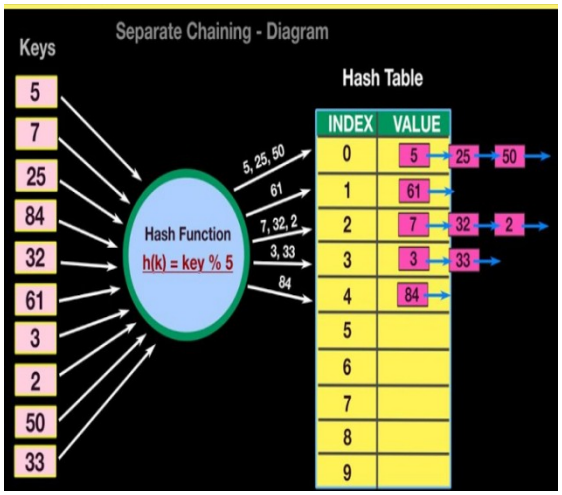
✔ Table never gets "full" (as long as memory exists)

Disadvantage:

✘ Extra memory required for pointers

✘ Worst case search becomes O(n)





2. Open Addressing

If collision occurs, find another empty slot.

i) Linear Probing: Search for empty slot sequentially.

Formula:

$$h(k, i) = [h(k) + i] \bmod m$$

Where

m = size of the hash table,
h(k) = (k mod m),
i = the probe number that varies from **0 to m–1**.

Therefore, for a given key k, the first location is generated by **[h(k) + 0]** mod m, the first time i=0. If the location is free, the value is stored at this location. If value successfully **stores** then probe count is 1 means location is founded on the first go.
If location is not free then second probe generates the address of the location given by **[h(k) + 1]mod m**. Similarly, if the generated location is occupied, then subsequent probes generate the address as **[h(k) + 2]mod m**, **[h(k) + 3]mod m**, **[h(k) + 4]mod m**, **[h(k) + 5]mod m**, and so on, until a free location is found.

Problem

- **Clustering:** One issue with linear probing is clustering, where a bunch of occupied spots clump together, slowing down the insertion and search processes. To minimize clustering, the table should have enough empty spots and use a good hash function that spreads items evenly.
- **Infinite Loops:** If the table gets full, linear probing could get stuck in an infinite loop. Ensuring the table never gets completely full or implementing a resizing mechanism can prevent this.

ii)**Quadratic Probing:** Search for empty slot in **quadratic steps** to reduce clustering.

Formula: $h(k, i) = (h(k) + i^2) \bmod m$

where **m** is the size of the hash table,
h(k) = (k mod m), i is the probe number that varies from **0 to m–1**,

- Quadratic probing solves the clustering problem which is in linear probing because instead of doing a linear search, it does a quadratic search.
- For a given key k, first, the location generated by [h(k) + 0] mod m, where i is 0. If the location is free, the value is stored at this generated location, else new locations will be generated using hash function [h(k) + 1^2] mod m.
- Value of i will change until free space is founded and probe count is increased until free space is founded.
  Quadratic probing performs better than linear probing, in order to maximize the utilization of the hash table.
- The disadvantage of quadratic probing is it does not search all locations of the list.

iii)Double Hashing: Search for empty slot using two hash functions.

Function: h(k, i) = [h1(k) + ih2(k)] mod m

where m is the size of the hash table,
h1(k) and **h2(k)** are two hash functions
h1(k) = k mod m & **h2(k)** = k mod m
i is the probe number that varies from **0 to m–1**
When we have to insert a key k in the hash table, we first probe the location given by applying [h1(k) mod m] because during the first probe, i = 0. If the location is vacant, the key is inserted into it. And if the location is not vacant then increase value of i to calculate next location using

h(k,1) = [h1(k) + 1*h2(k)] mod m

Best among open addressing techniques

✔ Avoids clustering

✔ Wide spread of keys

Setup (common to all)

Keys to insert (Nano Bananas 🍌): 23, 43, 13, 33, 55, 24
Hash table size (m): 10 *(indices 0 to 9)*
Primary hash function: `h(k) = k mod 10`

Empty hash table (initially)

0	1	2	3	4	5	6	7	8	9

---

1. LINEAR PROBING

Probe sequence: $h_i(k) = (h(k) + i) \bmod m \quad (i = 0, 1, 2, \ldots)$

Step 1: Insert 23
$h(23) = 23 \bmod 10 = 3$

0	1	2	3	4	5	6	7	8	9
			23 🍌						

Step 2: Insert 43
$h(43) = 43 \bmod 10 = 3$

0	1	2	3	4	5	6	7	8	9
			23 🍌	43 🍌					

Step 3: Insert 13
$h(13) = 13 \bmod 10 = 3$

0	1	2	3	4	5	6	7	8	9
			23 🍌	43 🍌	13 🍌				

Step 4: Insert 33
$h(33) = 33 \bmod 10 = 3$

0	1	2	3	4	5	6	7	8	9
			23 🍌	43 🍌	13 🍌	33 🍌			

Step 5: Insert 55
$h(55) = 55 \bmod 10 = 5$

0	1	2	3	4	5	6	7	8	9
			23 🍌	43 🍌	13 🍌	33 🍌	55 🍌		

Step 6: Insert 24
$h(24) = 24 \bmod 10 = 4$

0	1	2	3	4	5	6	7	8	9
			23 🍌	43 🍌	13 🍌	33 🍌	55 🍌	24 🍌	

Final hash table (Linear Probing)

0	1	2	3	4	5	6	7	8	9
-	-	-	23 🍌	43 🍌	13 🍌	33 🍌	55 🍌	24 🍌	-

---

2. QUADRATIC PROBING

Probe sequence: $h_i(k) = (h(k) + i^2) \bmod m \quad (i = 0, 1, 2, \ldots)$

Step 1: Insert 23
$h(23) = 23 \bmod 10 = 3$

0	1	2	3	4	5	6	7	8	9
			23 🍌						

Step 2: Insert 43
$h(43) = 43 \bmod 10 = 3$

0	1	2	3	4	5	6	7	8	9
			23 🍌	43 🍌					

Step 3: Insert 13
$h(13) = 13 \bmod 10 = 3$

0	1	2	3	4	5	6	7	8	9
			23 🍌	43 🍌			13 🍌		

Step 4: Insert 33
$h(33) = 33 \bmod 10 = 3$

0	1	2	3	4	5	6	7	8	9
		33 🍌	23 🍌	43 🍌			13 🍌		

Step 5: Insert 55
$h(55) = 55 \bmod 10 = 5$

0	1	2	3	4	5	6	7	8	9
		33 🍌	23 🍌	43 🍌	55 🍌		13 🍌		

Step 6: Insert 24
$h(24) = 24 \bmod 10 = 4$

0	1	2	3	4	5	6	7	8	9
		33 🍌	23 🍌	43 🍌	55 🍌		13 🍌	24 ← FIXED: Math e	

Final hash table (Quadratic Probing) (CORRECTED)

0	1	2	3	4	5	6	7	8	9
-	-	33 🍌	23 🍌	43 🍌	55 🍌	-	13 🍌	24 🍌	-

---

3. DOUBLE HASHING

Probe sequence: $h_i(k) = (h(k) + i \times h_2(k)) \bmod m$
Primary: $h(k) = k \bmod 10$ | Secondary: $h_2(k) = 7 - (k \bmod 7)$

Step 1: Insert 23
$h_2(23) = 7-(23 \bmod 7)=5$

0	1	2	3	4	5	6	7	8	9
			23 🍌						

Step 2: Insert 43
$h_2(43) = 7-(43 \bmod 7)=6$

0	1	2	3	4	5	6	7	8	9
			23 🍌						43 🍌

Step 3: Insert 13
$h_2(13) = 7-(13 \bmod 7)=1$

0	1	2	3	4	5	6	7	8	9
			23 🍌	13 🍌					43 🍌

Step 4: Insert 33
$h_2(33) = 7-(33 \bmod 7)=2$

0	1	2	3	4	5	6	7	8	9
			23 🍌	13 🍌	33 🍌				43 🍌

Step 5: Insert 55
$h_2(55) = 7-(55 \bmod 7)=1$

0	1	2	3	4	5	6	7	8	9
			23 🍌	13 🍌	33 🍌	55 🍌			43 🍌

Step 6: Insert 24
$h_2(24) = 7-(24 \bmod 7)=4$

0	1	2	3	4	5	6	7	8	9
			23 🍌	13 🍌	33 🍌	55 🍌		24 🍌	43 🍌

Final hash table (Double Hashing)

0	1	2	3	4	5	6	7	8	9
-	-	-	23 🍌	13 🍌	33 🍌	55 🍌	-	24 🍌	43 🍌

6. Differentiate between Static Hashing and Dynamic Hashing. Explain the working of Dynamic Hashing (Extendable hashing) with an example.

Feature	Static Hashing	Dynamic Hashing
Number of Buckets	Fixed	Variable
Table Size	Constant	Grows/Shrinks
Collision Handling	Difficult	Better
Performance	Decreases with growth	Remains efficient
Complexity	Simple	Complex
Suitable For	Small databases	Large databases

Dynamic Hashing

In **dynamic hashing**, the hash table size changes dynamically according to data growth.

- Buckets can expand
- Better collision handling

Efficient for large databases

Types of Dynamic Hashing

There are two major techniques.

1. Extendible Hashing
2. Linear Hashing

1. Extendible Hashing

Extendible Hashing uses a **directory** of pointers to buckets. The directory size can **double** when needed.

Important concepts:

- Hash value represented in binary
- Global Depth
- Local Depth
- Bucket Splitting
- Directory Doubling

Terms:

- **Hash Function:** Converts keys into binary values
- **Directory**: Array of pointers to buckets
- **Global Depth (GD):** Number of bits used to index the directory
- **Local Depth (LD):** Number of bits used to identify a bucket
- **Bucket Capacity**: Maximum keys a bucket can store

Global Depth (GD)

Number of bits used from the hash value to index the directory.

Example: `GD = 2`

Directory:

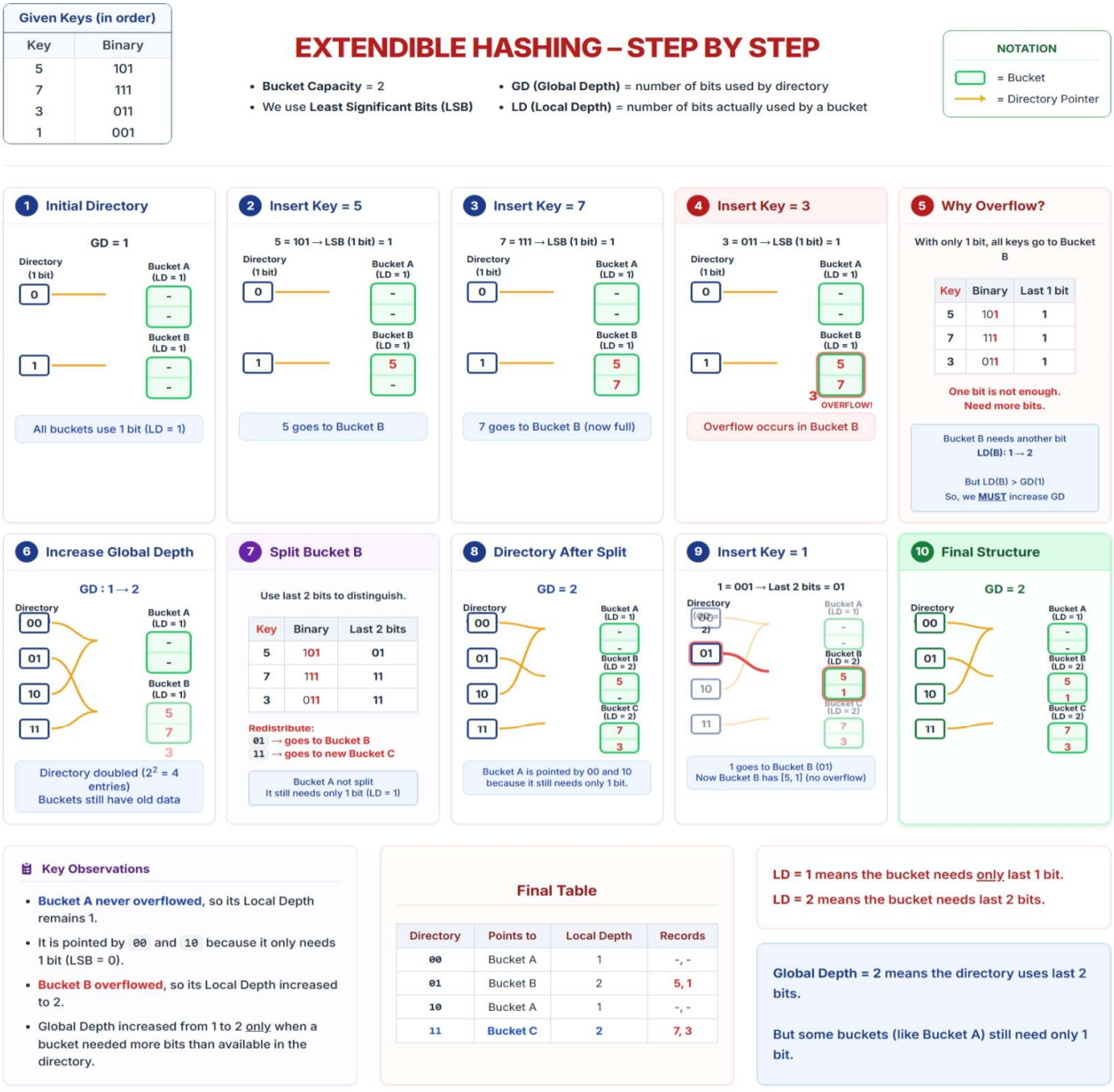
```
00
01
10
11
```

So there are $2^2 = 4$ directory entries.

Local Depth (LD)

The number of bits that a particular bucket currently uses to identify a bucket.

Always: `Local Depth ≤ Global Depth`



UNIT-II:

Q1 to 7 Refer Pdfs & do on your own

8. Explain the basic operations of a Binary Heap with their time complexities and also discuss the applications of Priority Queues.

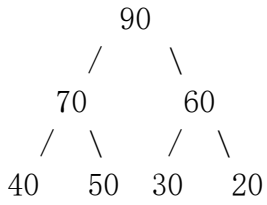
Binary Heap:

A **Binary Heap** is a **complete binary tree** that satisfies the **heap property**.

There are two types:

Type	Heap Property
Max Heap	Parent node ⩾ its children
Min Heap	Parent node ⩽ its children

Example: Max Heap



Here, every parent is greater than or equal to its children, so it is a **Max Heap**.

A binary heap is usually implemented using an **array**, because a complete binary tree does not require explicit pointers.

For a **0-indexed array**:

```
Parent(i) = (i - 1) / 2
Left(i)   = 2i + 1
Right(i)  = 2i + 2
```

2. Basic Operations of a Binary Heap

The major operations are:

1. Insertion
2. Deletion
3. Extract Min / Extract Max
4. Peek
5. Heapify
6. Build Heap

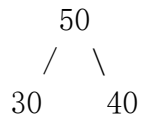
2.1 Insertion

A new element is always inserted at the **next available position** to maintain the complete binary tree property. Then the element is moved upward until the heap property is restored.

This process is called: `Heapify Up / Sift Up / Bubble Up`

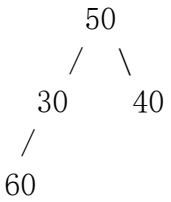
Example — Max Heap

Suppose we have:



Insert `60`.

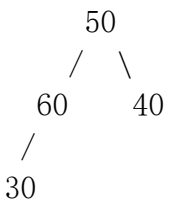
First place 60 at the next available position:



Now compare 60 with its parent:

$60 > 30$

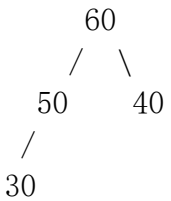
Swap:



Again compare with its parent:

$60 > 50$

Swap:



The heap property is restored.

Time Complexity

The newly inserted element can move from the bottom to the root.

The height of a binary heap is:$O(\log n)$

Therefore: Insertion=O(logn)

2.2 Deletion

In a binary heap, deletion generally refers to deleting the **root element**.

- In a **Max Heap**, delete the maximum element.
- In a **Min Heap**, delete the minimum element.

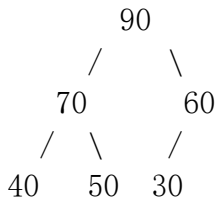
Steps

1. Remove the root.
2. Move the last element to the root.
3. Compare it with its children.
4. Move it downward until the heap property is restored.

This process is called: Heapify Down / Sift Down

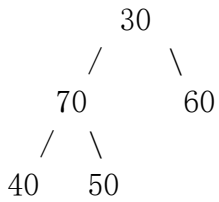
Example — Max Heap

Initial heap:



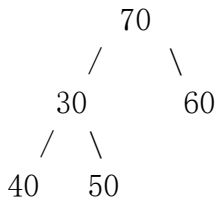
Delete 90.

Move the last element 30 to the root:



Compare 30 with its larger child, 70.

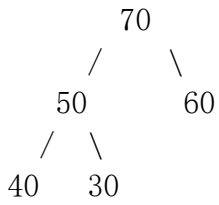
Swap:



Now compare 30 with its children.

The larger child is 50.

Swap:



Heap property is restored.

Time Complexity

The element may move from the root to the leaf.

Therefore: `Deletion=O(logn)`

2.3 Peek / Get Root

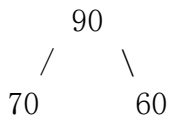
The root contains:

- **Maximum element** in a Max Heap.
- **Minimum element** in a Min Heap.

Therefore, we can access it directly.

Example

Max Heap:



`peek()` returns: 90

No restructuring is required.

Time Complexity: `O(1)`

2.4 Extract Max / Extract Min

This operation combines:

1. Reading the root.
2. Removing the root.
3. Restoring the heap property.

For a Max Heap: `extractMax()`

For a Min Heap: `extractMin()`

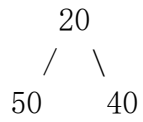
Time Complexity

Since heapify down may be required: `O(logn)`

2.5 Heapify

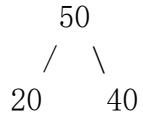
Heapify is the process of converting a subtree into a valid heap.

For example:



For a Max Heap, 50 should be above 20.

After heapify:



Heapify is mainly used during:

- Heap construction
- Deletion
- Heap Sort

Time Complexity: For one node: $O(\log n)$

2.6 Build Heap

Suppose we have an unsorted array: [20, 50, 10, 40, 30, 60]

We can convert it into a heap using **bottom-up heap construction**.

The important point is that we do **not** need to insert every element individually.

The Build Heap algorithm has: $O(n)$ time complexity.

3. Time Complexity Summary

Operation	Time Complexity	Reason
Access root / Peek	O(1)	Root is directly available
Insert	O(log n)	Heapify up
Delete root	O(log n)	Heapify down
Extract Min/Max	O(log n)	Remove + heapify
Heapify	O(log n)	Move element through tree height

Operation	Time Complexity	Reason
Build Heap	O(n)	Bottom-up construction
Search	O(n)	Heap is not a BST
Heap Sort	O(n log n)	Repeated extraction

4. What is a Priority Queue?

A **Priority Queue** is an abstract data type in which every element is associated with a **priority**.

The element with the highest priority is processed first.

Unlike a normal queue:

Normal Queue: First In → First Out

```
10 → 20 → 30 → 40
↑
First processed
```

Priority Queue

```
Element     Priority

A              3
B              1
C              5
D              2
```

If a higher number means higher priority:

```
C → A → D → B
```

The element with priority `5` is processed first.

5. Binary Heap and Priority Queue

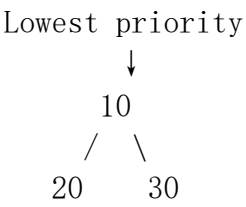
A **Binary Heap is one of the most efficient data structures used to implement a Priority Queue.**

For a **Max Heap**:

```
Highest priority
        ↓
       90
      /  \
    70    60
```

The highest-priority element is always at the root.

For a **Min Heap**:



The lowest-priority element is always at the root.

6. Applications of Priority Queues

Priority Queues have many practical applications.

6.1 CPU Scheduling

Operating systems can use priority-based scheduling.

Example:

Process	Priority
P1	2
P2	5
P3	1
P4	4

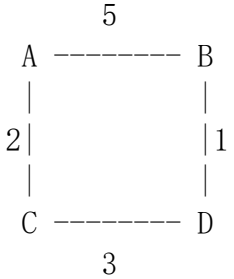
If a larger value means higher priority: P2 → P4 → P1 → P3

The highest-priority process gets CPU time first.

6.2 Dijkstra's Shortest Path Algorithm

Dijkstra's algorithm repeatedly selects the vertex with the **smallest tentative distance**.
A **Min Priority Queue** can efficiently identify this vertex.

Example:



The priority queue can contain:

```
(distance, vertex)
```

```
(2, C)
(5, B)
(6, D)
```

The vertex with the smallest distance is extracted first.

Therefore, priority queues help make Dijkstra's algorithm efficient.

6.3 Prim's Minimum Spanning Tree Algorithm

Prim's algorithm repeatedly selects the **minimum-weight edge** that connects a new vertex to the growing MST.

A Min Priority Queue can store candidate edges.

Thus:

```
Priority Queue
      ↓
Minimum-weight edge
      ↓
Add to MST
```

6.4 Huffman Coding

Huffman coding is used for **data compression**.

It repeatedly selects the two nodes with the smallest frequencies.

A Min Priority Queue is ideal for this operation.

Example:

Character	Frequency
A	5
B	9
C	12
D	13

The two smallest frequencies are repeatedly removed and combined.

```
5 + 9 = 14
```

```
12 + 13 = 25
14 + 25 = 39
```

A Min Heap makes selecting the smallest frequencies efficient.

6.5 Event Simulation

Priority Queues are used in simulations where events occur at different times.

Example:

Event	Time
E1	10
E2	4
E3	8
E4	2

The event with the earliest time should be processed first:

E4 → E2 → E3 → E1

A Min Priority Queue can efficiently manage these events.

6.6 Network Routing

Priority Queues can be used in network routing algorithms to select the next node or path with the smallest cost.

For example:

```
Router
  ↓
Select minimum-cost path
  ↓
Forward packet
```

This is particularly relevant to algorithms such as Dijkstra's shortest-path algorithm.

6.7 Job Scheduling

Suppose a system receives jobs:

Job	Priority
J1	3
J2	8
J3	1
J4	6

A Max Priority Queue can process: J2 → J4 → J1 → J3

This ensures high-priority jobs are processed first.

7. Priority Queue Applications — Summary

Application	Type of Priority Queue	Purpose
CPU Scheduling	Max/Min PQ	Select next process
Dijkstra's Algorithm	Min PQ	Select minimum-distance vertex
Prim's Algorithm	Min PQ	Select minimum-weight edge
Huffman Coding	Min PQ	Select lowest-frequency nodes
Event Simulation	Min PQ	Process earliest event
Job Scheduling	Max/Min PQ	Process highest/lowest priority job
Network Routing	Min PQ	Select minimum-cost path